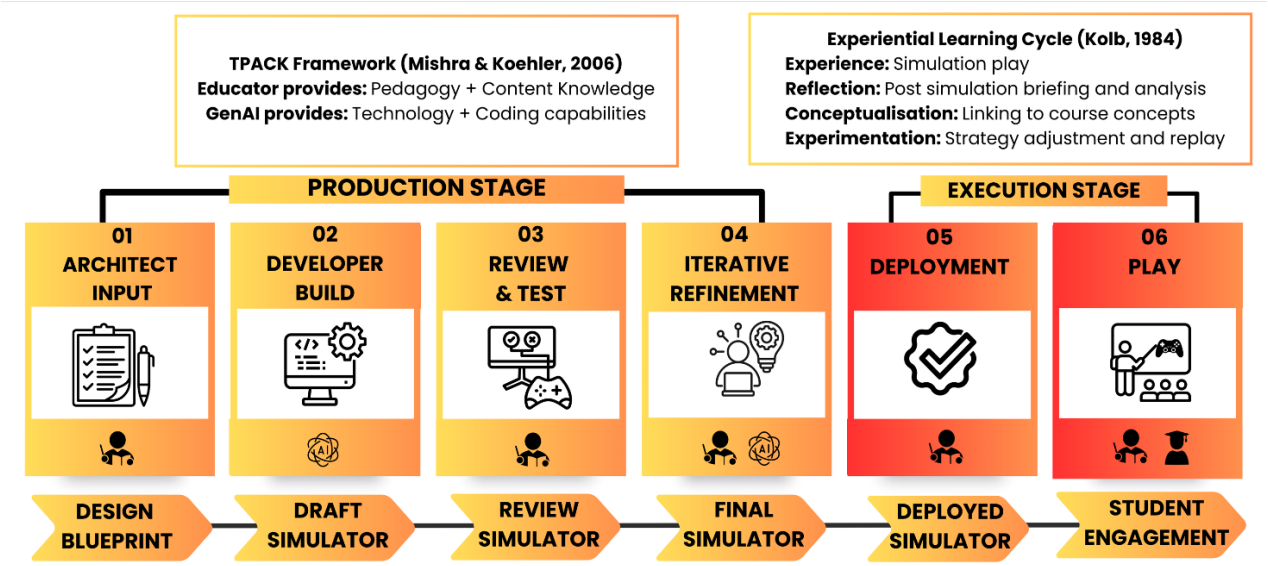
ADEPT

AI-Developed, Educator-Directed Pedagogical Transformation

TEACHER'S MANUAL AND REPLICATION GUIDE



The six phases of ADEPT, from educator input to student play.

Vincent Alexander Rangel · Muhammad Raees Shaik Saffarudin
Mika Tan Mei Jia · Mohamad Farhan Najmi Mohd Khairul Fauzi
Sunway College Johor Bahru, Malaysia

Version 1.0 · [24 August 2026] · vincentr@sunway.edu.my

INTRODUCTION

This manual takes an educator with no programming background from a single learning outcome to a working, browser-based simulator that students can play in class. Everything is done by typing instructions in plain language into a Generative AI tool. You will not write, read or edit a single line of code.

The guide follows the six phases of ADEPT in order. Work through it once with a real topic of your own in front of you, rather than reading it end to end first. Phases 1 to 5 build the simulator. Phase 6 runs it with students.

INTRODUCTION 1
1. ADEPT OVERVIEW 2
2. LEARNING OUTCOME IDENTIFICATION 4
3. PHASE 1 - ARCHITECT INPUT 5
4. PHASE 2 - DEVELOPER BUILD 8
5. PHASE 3 - REVIEW AND TEST 10
6. PHASE 4 - ITERATIVE REFINEMENT 13
7. PHASE 5 - DEPLOYMENT 15
8. PHASE 6 - PLAY 18
9. RESPONSIBLE USE AND TROUBLESHOOTING 20
10. ADAPTING ADEPT 21
APPENDIX A - PHASE PROMPT SHEET 22
APPENDIX B - PEDAGOGICAL ACCURACY CHECKLIST 23
APPENDIX C - GLOSSARY 24
APPENDIX D - FAQ 25
CITATION AND CONTACT 25
REFERENCES 25

Resources Required:

- **A Generative AI tool with a live preview panel:** The preview goes by different names: Canvas in ChatGPT and Gemini, Artifacts in Claude. Any tool that shows you the working game beside the conversation will do. A paid subscription is not essential, but free tiers limit how many long messages you can send in a day, and building a simulator takes many messages.
- **A web browser:** Chrome, Edge, Safari or Firefox.
- **Teaching material:** The learning outcome, the key concepts, any figures or tables students are expected to use, and a real-world situation the topic applies to.

1. ADEPT OVERVIEW

1.1 ADEPT's Output

The output of ADEPT is one file with a name ending in .html. Double-click it and it opens in a browser, exactly like a web page, except that everything the game needs is stored inside that one file: the rules, the scoring, the graphics, the text. Nothing is downloaded while it runs.

file works without an internet connection once students have it, which suits classrooms with weak Wi-Fi. It works on any device with a browser, including phones, so there is no minimum lab specification, and it can be copied, emailed or uploaded like any other document, so sharing it with a colleague cost nothing.

1.2 Key Roles

ADEPT splits the responsibilities into two key roles:

Role	Who fills it	Responsibilities
Architect	Educator	Decides the learning outcome, the scenario, what students decide, what counts as a good decision, what the simulator rewards, and whether the content is correct. Approves or rejects everything the Developer produces.
Developer	Generative AI	Proposes a design, writes all the code, builds the interface, implements changes on request, fixes fault it is told about. Decides nothing pedagogical.

The division comes from the TPACK framework (Mishra & Koehler, 2006). You hold the pedagogical knowledge and the content knowledge. The AI supplies the technological knowledge that has historically kept non-programming faculty out of simulation-based teaching. Keeping the boundary clean is what protects the quality of the result: when educators start accepting the AI's judgement about what should be taught, the simulator drifts away from the syllabus.

1.3 APDET Framework

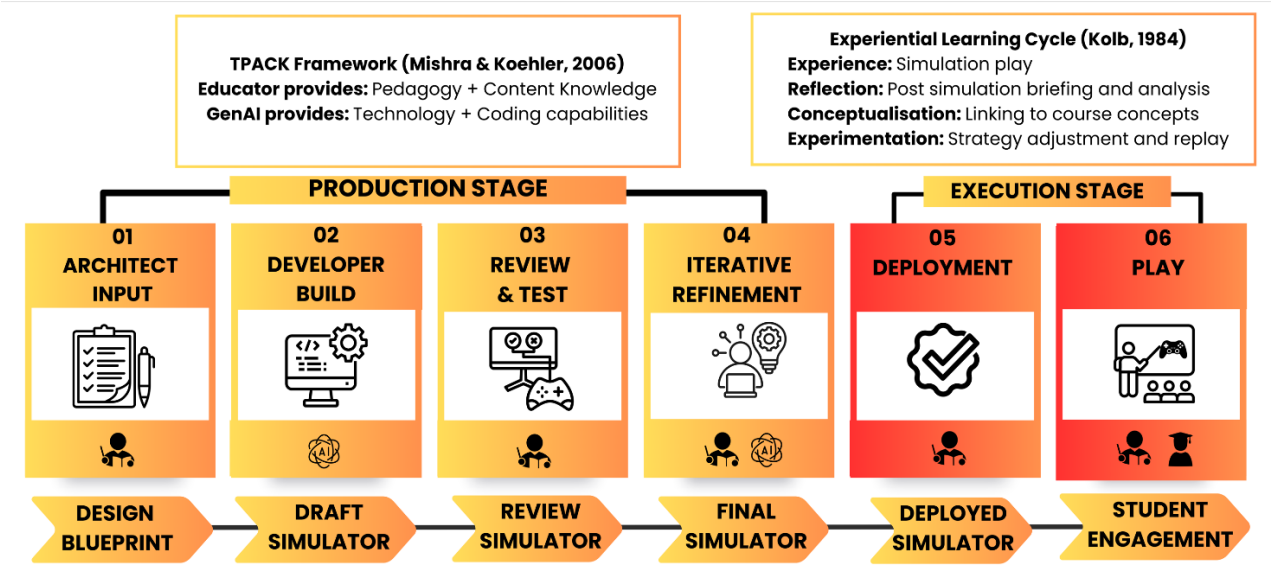


Figure A: The ADEPT framework. Phases 1 to 4 form the production stage; phases 5 and 6 form the execution stage.

Phase	Educator (Architect)	AI (Developer)	Output	Completion Stage (%)
Phase 1 *Architect input*	Complete the structured prompt template	Proposes design, mechanics and scoring	Design blueprint	10%
Phase 2 *Developer build*	Review the blueprint and approve the build	Generates a self-contained HTML file	Draft simulator	40%
Phase 3 *Review and test*	Play-test and score against the Pedagogical Accuracy Checklist	Revises on your instruction	Reviewed simulator	70%
Phase 4 *Iterative refinement*	Request improvements in plain language	Suggests and implements until you approve	Final simulator	90%
Phase 5 *Deployment*	Download the file and publish it	-	Deployed simulator	100%
Phase 6 *Play*	Facilitate, debrief and link back to concepts	-	Student learning	-

1.4 ADEPT Disclaimers

Four boundaries keep the method honest, and reviewers of teaching innovation will look for all of them.

- It is not automated course design. The AI never chooses the learning outcome, never decides what a correct answer is, and never signs off its own work.
- It is not a replacement for teaching. The simulator produces an experience. The learning happens in the debrief, when students connect their in-game decisions to the concepts you have taught.
- It is not a shortcut around subject expertise. Reviewing AI output for factual and conceptual error is the most demanding phase of the whole method, and it requires you to know the topic well.
- It is not free of failure. Generative AI produces confident, plausible, wrong numbers.

2. LEARNING OUTCOME IDENTIFICATION

2.1 Selecting the learning outcome

The most common cause of a disappointing first simulator is a learning outcome that is too broad or unclear. Hence, when selecting your lesson objective, it is best to be as detailed as possible for better pedagogy alignment. For example:

Too broad	Workable
Understand supply chain management	Evaluate the trade-off between cost and delivery speed when selecting a transport mode
Learn about marketing	Allocate a fixed advertising budget across channels to maximise reach within a target segment
Know facility location factors	Rank candidate sites against weighted location factors and justify the selection

2.2 Gather the lesson material

The AI builds the game around whatever content you give it. No providing any material may result in the AI inventing plausible-sounding content of its own, which may be different from what students learn or the learning outcome. Collect the following into one document or have the file ready to upload.

- The section of your lecture notes, slides or textbook chapter that covers the topic, including the exact terminology you use with students.
- Any table, formula or classification students are expected to apply, such as a comparison of transport modes, a costing formula or a set of weighted criteria.
- The numbers you want treated as authoritative: standard costs, lead times, capacities, percentages. Say explicitly that these are the figures to use.
- The vocabulary level of your students, particularly if English is their second language.

COPYRIGHT AND CONFIDENTIALITY

Upload only material you are entitled to share. Publisher slides, licensed case studies and copyrighted figures usually may not be uploaded to a commercial AI service. Summarise the concept in your own words instead or use openly licensed material. Never paste student names, student numbers, grades or any other personal data into a prompt. Use fictional company and character names throughout.

3. PHASE 1 - ARCHITECT INPUT

PHASE 1 OVERVIEW:

- **Purpose:** Turn your teaching intent into a design the AI can build from.
- **Educator provides:** A completed prompt.
- **AI produces:** Design blueprint in chat, with no code.
- **Deliverable:** Design blueprint (10% complete).

Step 1.1 Open a new conversation

Start a fresh chat rather than continuing an existing one. Old conversations carry old instructions, and the AI will blend them into your design without telling you. Keep this one conversation for the whole build: phases 1 to 4 all happen inside it, and the AI needs the earlier turns to remember what it has already agreed with you. You do not need the preview panel open yet. Phase 1 produces words, not code.

Step 1.2 Attach your lesson material

Upload the document from Section 2.2 or paste the relevant extract directly into the chat before sending the Phase 1 prompt. If you upload a file, refer to it by name in the prompt so the AI knows which parts matter, for example: "use section 9.1 of the attached notes".

Step 1.3 Complete and send the Phase 1 prompt

Copy the following prompt template below and replace each bracketed placeholder with your own wording. Keep the rest of the sentence structure as it is. The closing instructions are what stop the AI from jumping straight to code before you have agreed the design.

PROMPT TEMPLATE

*I want you to play the role of a game designer. I am thinking of making an HTML-based simulator-style game that focuses on **[learning objective]** for **[student level]**. This simulator focusses on **[scenario]** where players will **[brief game mechanic]**. Suggest to me how this can be designed, the detailed mechanics, core gameplay and scoring details. Use the **[lesson materials]** provided as the information to be integrated into the gameplay and details. Do not generate any code yet but give me the detailed mechanics, core gameplay and scoring details for me to review. Ask me if there are revisions to be made and once I give you the approval to generate the actual code, generate it in canvas.*

Placeholder	Educator's Input	Example
[learning objective]	The learning outcome from Section 2.1, in the words you would use with students	Learning the key factors for transportation mode selection in logistics
[student level]	Student programme and year to ensure AI designs the difficulty and language correctly	Business diploma students, second semester
[scenario]	The scenario you are aiming to simulate in the simulator	Managing their logistics operations as a logistics manager

Placeholder	Educator's Input	Example
[brief game mechanic]	The repeated decision to be done when students use the simulator and its consequence	Choose the appropriate shipping mode, adjust key parameters, and navigate potential in-transit issues by making choices that affect delivery time and cost until all items are successfully shipped
[lesson materials]	The exact section or file name you uploaded	9.1 Modes of Transportation and Their Major Issues (9.1.1 to 9.1.6)

EXAMPLE — LOGISTICS TYCOON SIMULATOR

Learning objective: Evaluate the key factors for transportation mode selection in logistics, such as cost and sustainability considerations.

I want you to play the role of a game designer. I am thinking of making an HTML-based simulator-style game that focuses on **learning the key factors for transportation mode selection in logistics** *for* **business diploma students**. *This simulator focusses on* **managing their logistics operations as a logistics manager** *where players will* **choose the appropriate shipping mode, adjust key parameters, and navigate potential in-transit issues by making choices that affect delivery time and cost until all items are successfully shipped**. *Suggest to me how this can be designed, the detailed mechanics, core gameplay and scoring details. Use the* **9.1's Modes of Transportation and Their Major Issues (9.1.1 to 9.1.6)** *provided as the information to be integrated into the gameplay and details. Do not generate any code yet but give me the detailed mechanics, core gameplay and scoring details for me to review. Ask me if there are revisions to be made and once I give you the approval to generate the actual code, generate it in canvas.*

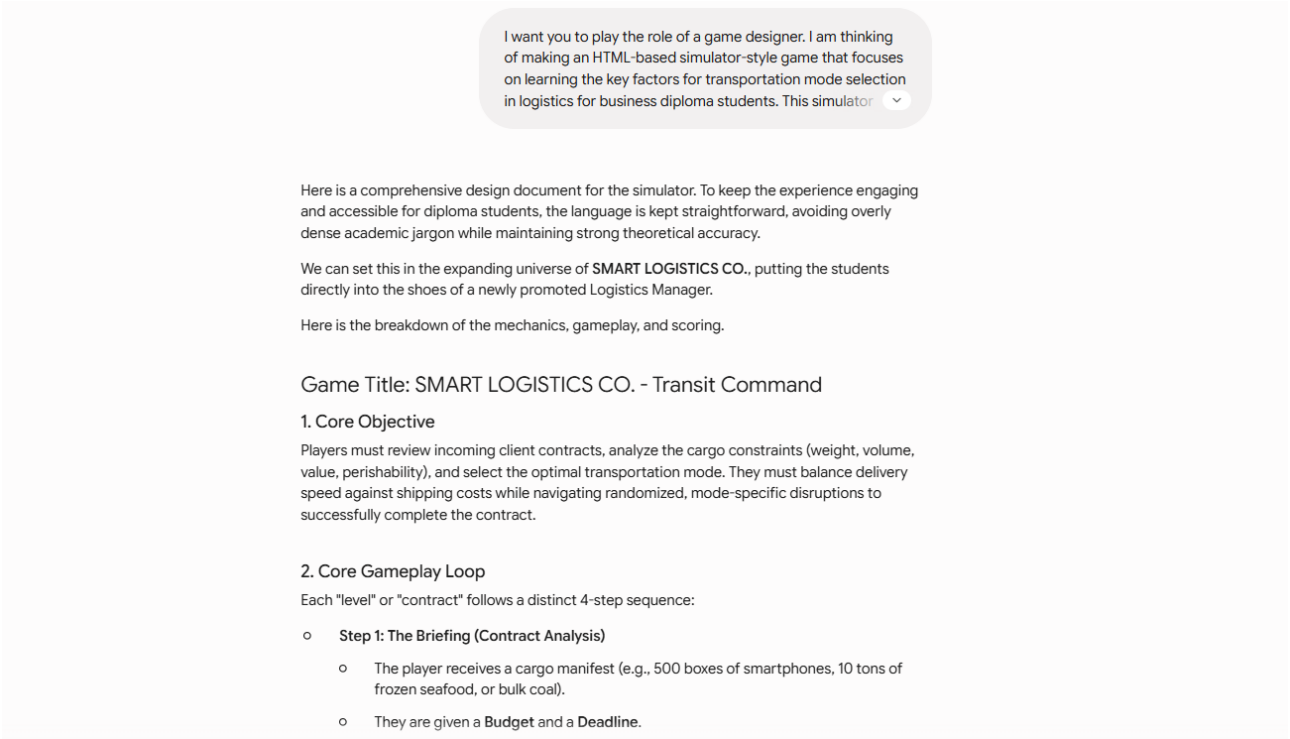


Figure 1: The design blueprint returned in Phase 1 for the Logistics Tycoon simulator.

Step 1.4 Assess the design blueprint

The AI will reply with a written design. Read it as you would read a colleague's lesson plan, and check that all six of the following are present and correct. Anything missing here becomes a defect in the finished simulator.

EXAMPLE — WHAT THE DESIGN BLUEPRINT CONTAINS (ABRIDGED ILLUSTRATION)

A usable blueprint for Logistics Tycoon covered the following ground, in the AI's own words and at greater length:

Premise. The player runs outbound logistics for an electronics distributor and must clear twelve shipments across four in-game weeks.

Core loop. Each round presents one shipment with a destination, weight, value and customer deadline. The player selects road, rail, sea or air, then sets two parameters: insurance level and consolidation.

Variables from the lesson material. Cost per tonne-kilometre, transit time, reliability and carbon output for each of the six modes in section 9.1.

Random events. Port congestion, fuel surcharge, customs hold and weather delay, each linked to the mode most exposed to it.

Scoring. Total profit (40%), on-time delivery rate (35%), carbon score (25%), with a penalty for any shipment that misses its deadline.

Feedback. After each shipment, a one-line explanation naming the factor that decided the outcome.

Length. Twelve rounds, roughly 12 to 15 minutes for a first-time player.

Step 1.5 Revise the blueprint (if necessary)

If the blueprint…	Reply with…
Drifts away from your learning outcome	"The scoring rewards speed more than the cost-versus-service trade-off I am teaching. Rebalance the scoring so the trade-off decides the winner and show me the revised scoring only."
Uses invented figures instead of yours	"Replace all cost and transit-time figures with the ones in the attached notes. List the figures you will use so I can verify them before we continue."
Is too complicated for the class time	"This is too long for a 15-minute classroom activity. Reduce it to eight rounds and remove the insurance parameter, keeping everything else the same."
Is thin or generic	"Give me two alternative designs for the same learning objective, each with a different core mechanic, so I can choose."

Repeat until the blueprint reads like something you would be willing to defend to a colleague. Only then move to Phase 2.

4. PHASE 2 - DEVELOPER BUILD

PHASE 2 OVERVIEW:

- **Purpose:** Turn the agreed blueprint into a working file.
- **Educator provides:** Approval and one build instruction.
- **AI produces:** Self-contained HTML simulator.
- **Deliverable:** Draft simulator (40% complete).

Step 2.1 Open the code preview panel

Stay in the same conversation and switch on the live preview.

- In ChatGPT: Open Canvas
- In Gemini: Open Gemini
- In Claude: Open Artifacts

The panel opens beside the chat and shows the raw code/game preview running while the AI writes it, so you can click through the simulator without downloading anything. If your tool has no preview panel, the AI will output only the raw code instead, and you can still build the file using the manual route in Section 7.1.

Step 2.2 Give the build instruction

Copy the following prompt template below and replace each bracketed placeholder with your own wording.

PROMPT TEMPLATE
*The proposed game overview and details are ok. Turn this into a playable HTML offline game using canvas that is suitable to be played on **[devices]**.*
*Build it as a single self-contained .html file. Put all the code, styling and content inside that one file and do not link to anything on the internet, so it runs with no connection. Use clear labels and readable font sizes for **[student level]**. Show me the working game in canvas when it is ready.*

Placeholder	What to write	Explanation
[devices]	"laptops and mobile phones", or "laptops only", or "a projector and laptops"	Determines the layout. A game designed for a laptop is often unusable on a phone, and half of a typical class will play on a phone.
[student level]	The same wording you used in Phase 1	Controls vocabulary and the amount of on-screen text.

EXAMPLE — LOGISTICS TYCOON SLIMUATOR, THE BUILD INSTRUCTION

The proposed game overview and details are ok. Turn this into a playable HTML offline game using canvas that is suitable to be played on **laptops and mobile phones**.

Build it as a single self-contained .html file. Put all the code, styling and content inside that one file and do not link to anything on the internet, so it runs with no connection. Use clear labels and readable font sizes for **business diploma students whose first language is not English**. *Show me the working game in canvas when it is ready.*

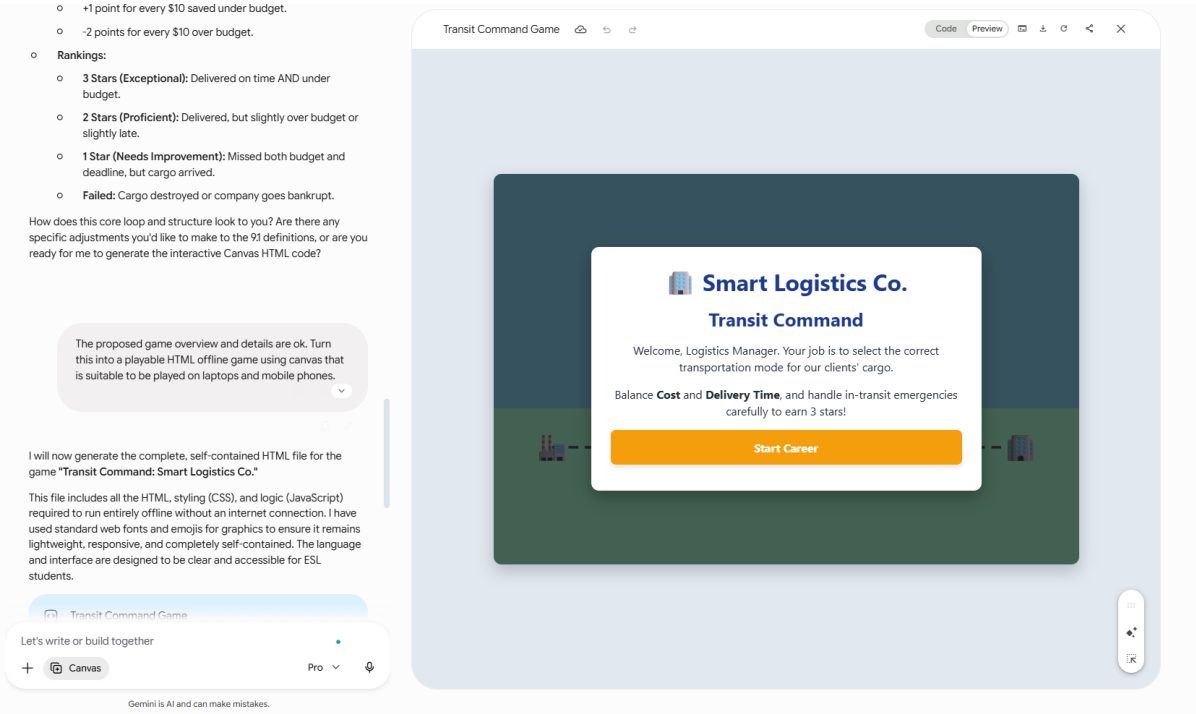


Figure 2: The draft simulator produced in Phase 2, running in the AI tool's preview panel.

Step 2.3 Run a general quick test

Generation takes anywhere from thirty seconds to several minutes, and the preview may look broken while the code is still being written. Wait until the AI says it has finished before judging anything. Then play one full round and check five things. This is a quick test for ensure the simulator works. Below are the things to be tested in this phase:

1. The game starts and shows an opening screen or a first decision.
2. Every button responds to a click, and nothing is cut off at the edge of the panel.
3. Making a decision changes something visible: a score, a budget, a message.
4. You can reach the end of one round and see a result.
5. You can restart or continue without reloading the page.

IF THE PREVIEW IS BLANK OR FROZEN

Ask for a rebuild in one message rather than a series of small fixes: *"The canvas is blank and nothing happens when I click Start. Please check the code for errors and rebuild the game so it runs. Keep the same design we agreed."* If the second attempt also fails, ask the AI to build a simplified version with the core loop only, then add the rest back during Phase 4.

5. PHASE 3 - REVIEW AND TEST

PHASE 3 OVERVIEW:

- **Purpose:** Confirm the simulator is factually correct and actually teaches the outcome.
- **Educator provides:** Completed Pedagogical Accuracy Checklist and a fault log.
- **AI produces:** Updates simulator based on educator's instruction.
- **Deliverable:** Reviewed simulator (70% complete).

This phase is the safeguard that ensures the ADEPT's output is pedagogically aligned. The educator serves as the quality control between a generated file and a classroom of students.

Step 3.1 Test the simulator

Perform three different play-through of the rough simulator where each focuses on a different intention.

Intention	What you do	What it reveals
Play Through 1: *The good student*	Play the way you would want a student to play, applying the concepts correctly at every decision	Whether the simulator rewards correct reasoning. If careful play does not produce a good score, the scoring is wrong and the game will teach the opposite of what you intend.
Play Through 2: *The lazy student*	Click the easiest or first option every time without reading anything	Whether a mindless strategy can win. If it can, students will find it in five minutes and stop thinking. This is the most common fault in a first draft.
Play Through 3: *The breaker*	Enter zero, enter a huge number, click twice, go back, refresh, resize the window, open it on a phone	Whether the game crashes, produces negative values, allows a score of infinity, or becomes unreadable on a small screen which may break the simulator.

Step 3.2 Review simulator using the Pedagogical Accuracy Checklist

After reviewing the simulator, use the checklist to evaluate the simulators. Score each dimension out of 5, then calculate the weighted total. Appendix B is a printable blank copy.

Dimension	Weight	Scoring Focus
Content accuracy	**35%**	• Are the facts, figures, terminology and business logic correct and consistent with your teaching material? • Check every number that appears on screen against your notes. This is where hallucinated content hides.
Learning-objective alignment	**40%**	• Does playing the game teach the outcome you specified? • Would a student who scores well have demonstrated the reasoning in your syllabus? • This dimension carries the most weight because a polished simulator that teaches the wrong thing has negative value.
Game variables	**25%**	• Do the mechanics, scoring and difficulty behave sensibly? • Is the challenge pitched for your cohort? • Do rewards follow the decisions you want to encourage?

Scoring Descriptors

Score	Meaning
5	No issues found. Ready to use as it is.
4	Minor issues that do not affect learning. Cosmetic wording or layout.
3	Noticeable issues. A student could form a wrong impression. Fix before deployment.
2	Serious issues. Incorrect content or scoring that contradicts the outcome.
1	Unusable. Rebuild the affected part from the blueprint.

Weighted score = (0.35 × content) + (0.40 × alignment) + (0.25 × variables)

- **If weighted score is above 4.0**: *Proceed to Phase 4.0 for iterative refinement.*
- **If weighted score is between 3.0 to 4.0:** *Perform 5.3 to 5.5 to fix any severe pedagogical alignment issues before proceeding to Phase 4.0.*
- **If weighted score is below 3.0:** *Return to Phase 2.0 rather than patching the build, because the problem is usually in the design rather than the code.*

Step 3.3 Keep an issue log

Record down each issues you find rather than fixing them as they appear. Fixing while testing produces a build where each repair breaks something already checked. Below is an example of or an issue log:

Where	What happened	What should happen	Priority
Round 3, air freight option	Cost shown as RM 420 per tonne	RM 4,200 per tonne, as in section 9.1.3 of the notes	High
Final score screen	Carbon score is displayed but not included in the total	Carbon contributes 25% of the final score	High
Any round	Choosing sea freight every time wins comfortably	No single mode should win every scenario	High
Mobile view	The decision buttons run off the right of the screen	All buttons visible on a phone screen	Medium
Feedback message	"Good job!" with no explanation	One line naming the factor that decided the outcome	Medium

Step 3.4 Report Issues to the AI precisely

Report one fault at a time using three pieces of information: where it happens, what happens, and what should happen instead. Then contain the change.

> ***PROMPT TEMPLATE***
> *In **[where in the game]**, the current behaviour is **[what happens now]**. It should be **[what should happen]**.*
> *Please fix only this. Do not change any other mechanic, the scoring weights, the layout or the wording anywhere else in the game. After the fix, tell me in one sentence what you changed.*

EXAMPLE — A FAULT REPORTED IN LOGISTICS TYCOON

In **the air freight option in every round**, *the current behaviour is* **that the cost is shown as RM 420 per tonne**. *It should be* **RM 4,200 per tonne, matching section 9.1.3 of the notes I uploaded**.

Please fix only this. Do not change any other mechanic, the scoring weights, the layout or the wording anywhere else in the game. After the fix, tell me in one sentence what you changed.

Step 3.5 Re-test updated simulator fix

Generative AI regularly repairs one thing and disturbs another without mentioning it. After each fix, replay the part that was repaired and the part immediately connected to it, usually the score. Once all the fixes have been verified, re-review the updated simulator by repeating 5.1 and 5.2 before moving to Phase 4.0 Iterative Refinement

Pedagogy Accuracy Review Checklist

To systematically evaluate whether the AI-generated simulator aligns with pedagogical intent and expectations.

SUBJECT CODE & NAME	PROGRAMME / LEVEL	SIMULATOR NAME	REVIEW DATE
BITB2083 Global Supply Chain Management	Diploma	Logistics Tycoon Simulator	14 Feb 2026

LEARNING OBJECTIVE — LO1	LEARNING OBJECTIVE — LO2
Evaluate the key decisions for each transportation mode in logistics	-

How to Use — Rate each criterion from ① Not Aligned to ⑤ Fully Aligned.

A	SECTION A · CONTENT ACCURACY						WEIGHT 35%
CRITERIA		1	2	3	4	5	SCORE
Conceptual accuracy aligns with syllabus		①	②	③	④	✓⑤	5
Appropriate depth for student level		①	②	③	✓④	⑤	4
Correct use of terminology		①	②	③	✓④	⑤	4
Game rules reflect real-world principles		①	②	③	④	✓⑤	5
SECTION A AVERAGE SCORE							4.50

B	SECTION B · LEARNING OBJECTIVE ALIGNMENT						WEIGHT 40%
CRITERIA		1	2	3	4	5	SCORE
All LOs are represented in gameplay		①	②	③	④	✓⑤	5
Game mechanics operationalise LOs		①	②	③	④	✓⑤	5
Scoring reflects LO achievement		①	②	③	④	✓⑤	5
Matches intended taxonomy level		①	②	③	④	✓⑤	5
SECTION B AVERAGE SCORE							5.00

C	SECTION C · GAME VARIABLES						WEIGHT 25%
CRITERIA		1	2	3	4	5	SCORE
Realistic business scenario		①	②	③	④	✓⑤	5
Variables appropriate for student level		①	②	③	✓④	⑤	4
Logical and consistent gameplay		①	②	③	④	✓⑤	5
Engagement does not compromise accuracy		①	②	③	④	✓⑤	5
SECTION C AVERAGE SCORE							4.75

FINAL SCORING

Section	Avg	Weight	Weighted
Section A. Content Accuracy	4.50	0.35	1.6
Section B. LO Alignment	5.00	0.40	2.0
Section C. Game Variables	4.75	0.25	1.2
FINAL SCORE			4.8

SIMULATOR QUALITY GUIDE

Score	Simulator Quality
> 4.5	• Excellent — Ready for Phase 4
> 4.0	• Good — Minor refinement
> 3.5	• Moderate — Revision needed
< 3.5	• Poor — Redesign required

KEY STRENGTHS	KEY ISSUES IDENTIFIED	RECOMMENDED REVISIONS
Mobile friendly interface with customisable parameters and randomised events	Game can be complex for first time players requiring lecturers to explain simulator mechanics	Add a short explanation tab that explains each tab, feature and simulator operating steps before students start the simulator.

Figure 3: A completed Pedagogical Accuracy Checklist from the Phase 3 review

6. PHASE 4 - ITERATIVE REFINEMENT

PHASE 4 OVERVIEW

- **Purpose:** Refine the simulator from being pedagogically accurate to be engaging and ready for deployment.
- **Educator provides:** Improvement requests or suggestions in plain language.
- **AI produces:** Suggestions and implementations based on educator's instructions.
- **Deliverable:** Final simulator (90% complete).

Step 4.1 Request AI to propose refinements

A useful move is to have the Developer suggest options and the Educator review an selects the recommendations to be integrated. This allows for the educator to be able to improve the engagement and gamification of the simulator while maintaining the pedagogical alignment that was established in Phase 3.0.

PROMPT TEMPLATE

How to improve the game mechanics and game logic so that it better matches the learning objective and learning materials? Provide some suggestions on how to improve them for me to review and choose which to implement into the game. I will let you know which suggestion to be integrated into the game.

Expect six to ten suggestions. Select the suggestions that best fits the simulator objective, student level and device used, reject the rest explicitly, and mention why it was rejected in a simple sentence. The reason matters as it teaches the AI what you value, and later suggestions in the same conversation improve as a result.

Below are some examples of refinements that could be considered:

Refinement	Why it matters for learning	Example of instruction to send
Explain every outcome	A result without a reason teaches guessing. The explanation is where the concept is delivered.	"After each decision, show a two-line explanation naming the factor that decided the outcome and the concept it illustrates. Keep the language simple."
Remove the dominant strategy	If one option always wins, the trade-off you are teaching disappears.	"Sea freight currently wins in every scenario. Adjust the scenario variables so that each mode is the best choice in at least two rounds and tell me which rounds favour which mode."
Add a visible running total	Students self-correct when they can see the effect of the last decision.	"Display the budget, on-time rate and carbon score at the top of the screen at all times and highlight the one that just changed."
Give an end-of-game summary	The summary is what students bring to the debrief.	"At the end, show a summary of every decision, its result and the final weighted score, formatted so a student can screenshot it."
Fix the accessibility	Some students play on a small phone in a bright room.	"Use high-contrast colours, a minimum 16-pixel font, and make every control reachable with the keyboard. Check the layout works on a 6-inch phone screen."
Add a second scenario	Fast finishers need somewhere to go, and a second dataset tests transfer rather than recall.	"Add an optional hard mode using a different set of orders and tighter deadlines, selectable from the start screen."

Step 4.2 Change one thing at a time

Send one instruction, wait, test, save, then send the next. Batching five changes into one message reliably produces a build where two of them work, one is misunderstood, and two silently break something else.

The single most useful sentence to append to any change request is: "Change only this and keep everything else exactly as it is."

Save a numbered version of the file after every change you accept. When a refinement makes the game worse, you can return to the previous version instead of trying to talk the AI backwards, which rarely works.

Long conversations cause hallucinations due to conversation limits. After a few hours of work the AI starts forgetting earlier decisions, reintroducing faults you fixed, or rewriting sections it was told to leave alone.

If that begins, open a new chat and hand over the file rather than persisting and use the following prompt below to continue refinement.

> ***PROMPT TEMPLATE***
> *I have the following HTML game attached which I want you to refine. The complete code is below. Read it and confirm you understand the game's rules and scoring before changing anything.*
> *This game teaches* **[learning objective]** *to* **[student level]**. *The scoring works as follows:* **[one-line summary]**.
> *Do not redesign or rewrite it. I will give you one change at a time, and you will implement only what I ask, keeping everything else identical. First, confirm what you understand about the game.*
> *[Upload the latest revision of your saved .html file here]*

Refinement has no natural end point, and a simulator can absorb a whole weekend. Stop when all of the following are true and put the remaining ideas into a list for the future revisions.

- A colleague who does not teach the subject can play it without asking you a question.
- Someone playing badly on purpose scores clearly worse than someone playing well.
- One full play-through fits the time you have in the lesson, with the debrief still to come.

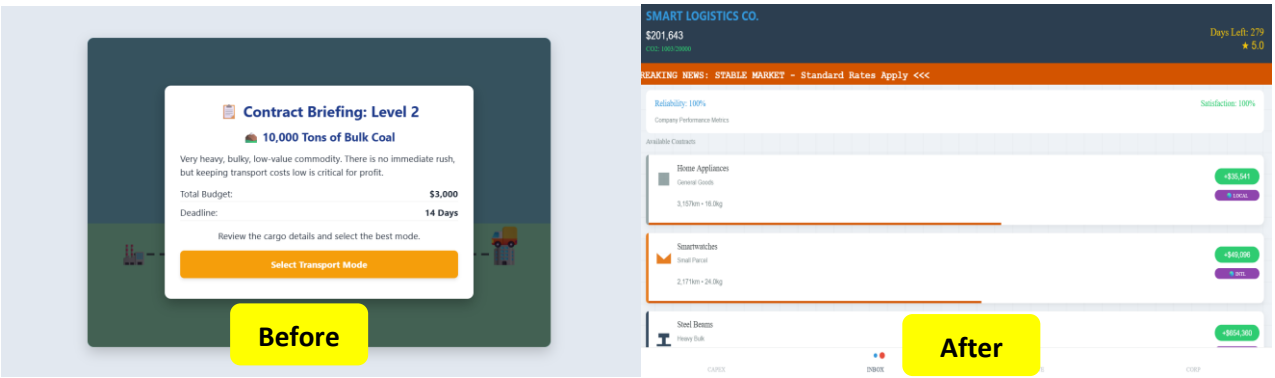


Figure 4: The same simulator screen before and after Phase 4 refinement.

7. PHASE 5 - DEPLOYMENT

PHASE 5 OVERVIEW

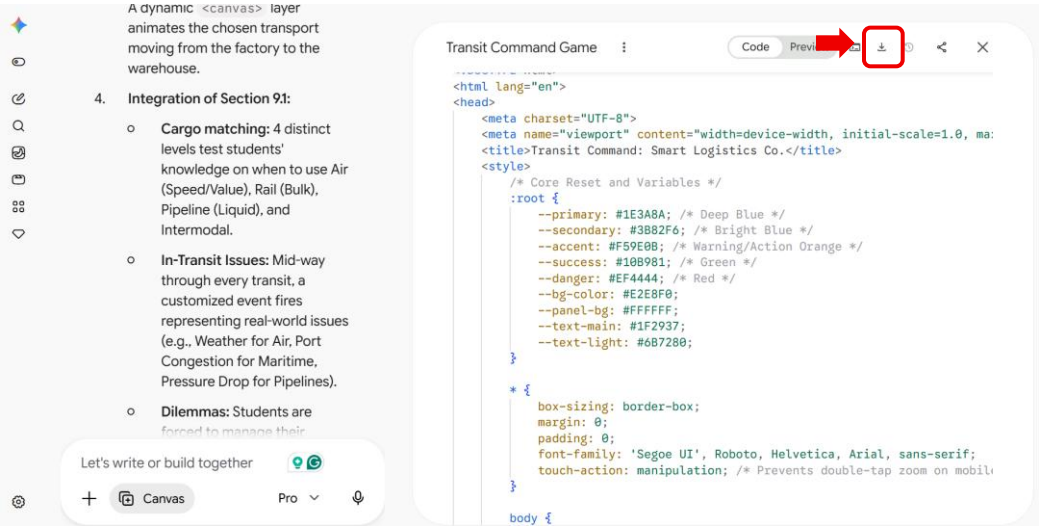
- **Purpose:** Download the finished file and publish it on hosting platform.
- **Educator provides:** A published link or a distributed file.
- **Deliverable:** Deployed simulator (100% complete).

Step 5.1 Download the final simulator

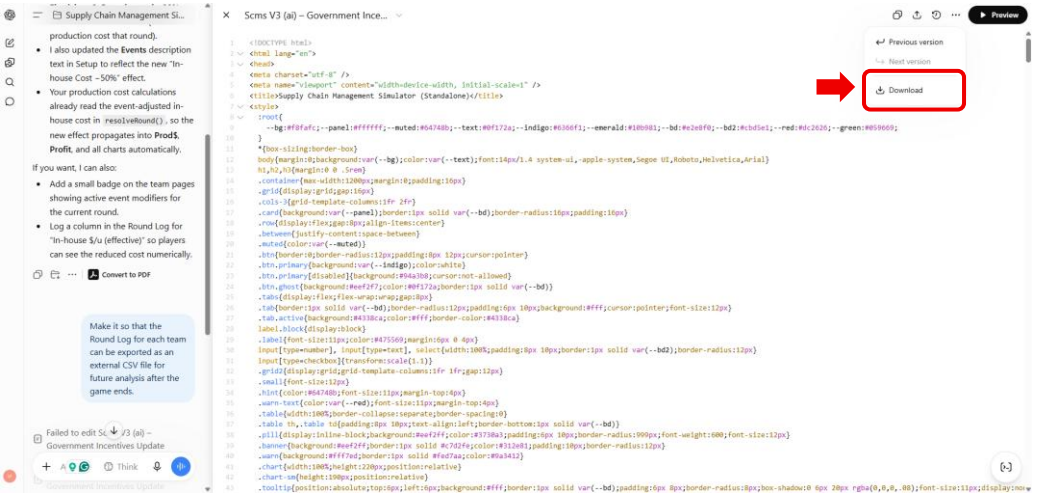
Option A: The download button

Most preview panels have a download or export option, often behind a menu icon in the top corner of the panel. Download the file and check that the name ends in .html. If it downloads as .txt, rename it and replace the extension.

Example: Gemini



Example: ChatGPT



Option B: Copy the code into a text file

If there is no download option:

1. Copy the complete code of the final version in one block.
2. On Windows, open Notepad. On Mac, open TextEdit and choose Format, then Make Plain Text.
3. Paste the code in.
4. Save the file. In the save dialogue, set the file name to **'simulator name'.html** and set the file type to All Files.
5. Double-click the saved file. It should open in your browser and run.

Step 5.2 Final simulator testing

The preview panel and the downloaded file are not the same thing, and a game that ran in the panel can fail once saved. Run three checks before publishing.

Check	How	If it fails
It opens on your own computer	Double-click the saved file	Re-download, or use Route B above
It works with no internet	Turn off Wi-Fi, then open the file again and play a round	Ask the AI: "Inline every library, font and image so this single HTML file runs with no internet connection at all."
It works on a student device	Send the file to your own phone and play one round	Ask the AI to make the layout responsive and re-test

Step 5.3 Publishing completed simulator

Platform	How it works	Best for	Watch out for
LMS file upload	Upload the .html file to Moodle, Canvas, Google Classroom or Blackboard as a file resource	Most classes, and any activity students revisit outside class	Some LMS settings download the file rather than opening it. Tell students to open the downloaded file from their device.
Free static hosting	Drag the file into a free hosting service to get a web link students can click	Large classes and phone-based play, where a link is easier than a file	Check your institution's policy on hosting teaching material externally. Anyone with the link can open it.
Direct distribution	Email the file, share it in a class group, or copy it from a USB drive	Low-bandwidth settings and computer labs with no internet	Email systems sometimes block .html attachments. Put the file in a zip folder if it is rejected.
Classroom projector	Open the file on your own machine and play as a whole class with students directing your choices	Introducing the simulator, or classes where devices are limited	One student ends up making all the decisions. Rotate who chooses each round.

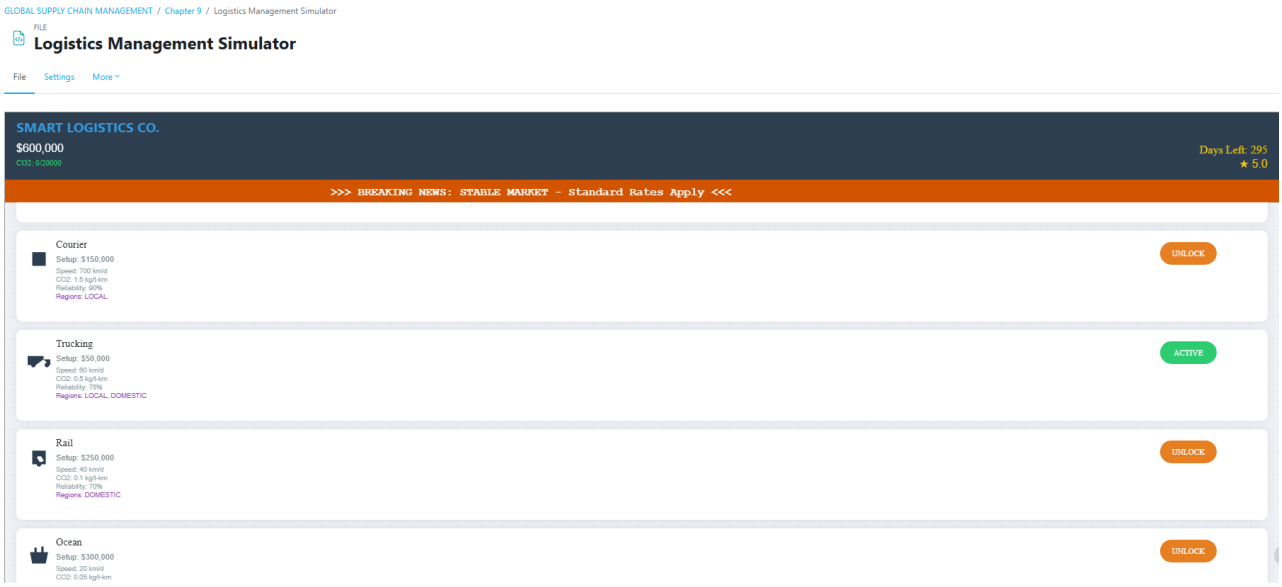


Figure 5: The deployed simulator as students sees it in the LMS platform.

8. PHASE 6 - PLAY

PHASE 6 OVERVIEW:

- **Purpose:** convert play into learning.
- **Educators:** Facilitate, debrief and a link back to the syllabus.
- **Deliverable:** Student engagement and learning outcomes.
- **Estimated Time:** 30 to 60 minutes of class time.

The simulator is the easy half. The learning is produced by what happens around it, which is why this phase follows Kolb's (1984) experiential learning cycle rather than simply allocating class time to gameplay. A session that stops at the end of the game delivers enjoyment and very little transfer.

Step 6.1 Lesson plan development

Time	Stage and educator's task	Student deliverable
0–5 mins	**Set up.** State the learning outcome, the scenario and the scoring rule. Confirm every student can open the file. Do not explain the strategy.	-
5–10 mins	**Demonstrate.** Project the simulator and play one round aloud, saying why you choose what you choose. Make one deliberate mistake and let them see the consequence.	-
10–25 mins	**Play (concrete experience).** Students play individually or in pairs against a target score. Circulate and ask one question per group rather than giving advice.	A score and a set of choices they can account for
25–33 mins	**Debrief (reflective observation).** Structured discussion. Take two contrasting scores and ask both students to narrate their reasoning.	Three written reflection answers
33–40 mins	**Connect (abstract conceptualisation).** Map their decisions to the concepts, using their language first and the syllabus terminology second.	Notes connecting each decision to a named concept
40–45 mins	**Replay (active experimentation).** Students play again with a revised strategy and compare the two scores.	An improved score and a one-line explanation of what changed

Step 6.2 Facilitating the play sessions

While students play, resist solving their problems. Instead provide them with guiding questions that move thinking forward without giving the answer away. Below are examples of guiding questions that could be asked:

- What are you optimising for right now, and is that the same thing the score is measuring?
- You lost points on that round. What did the game tell you about why?
- If you had half the budget, would that decision change?
- What have you not tried yet?

Step 6.3 Debrief session question bank

- What was your strategy, and how did it change as you played?
- Which single decision had the biggest effect on your score, and why?
- Where did a trade-off in the game mirror a trade-off a real manager faces?
- What did the game reward that you did not expect it to reward?
- If you replayed with what you know now, what one thing would you do differently?
- Which concept from this topic does the simulator help you understand better, and which does it oversimplify?

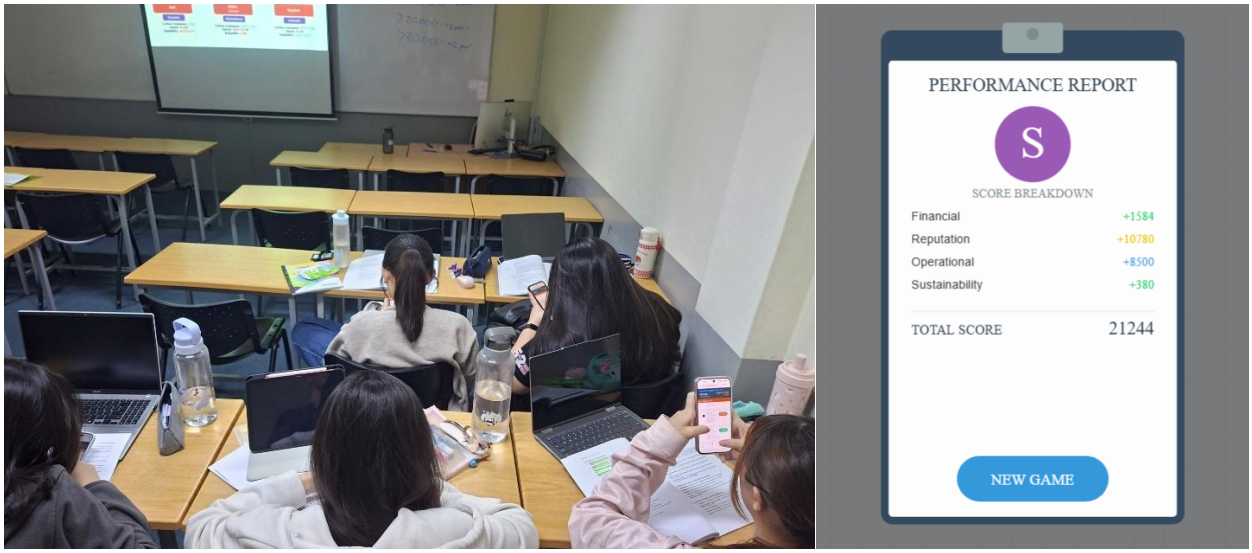


Figure 6: Students playing an ADEPT simulator during the Phase 6 classroom session.

9. RESPONSIBLE USE AND TROUBLESHOOTING

9.1 Responsible AI use disclaimer

- No AI-generated content should reach students without passing the Phase 3 review. This is not a formality; it is the condition under which the method is defensible.
- Generative AI produces plausible figures with complete confidence. Educator should verify costs, rates, capacities and formulas against your own material.
- Never paste student names, identification numbers, grades or any other personal data into a prompt. Use fictional companies and characters.
- Upload only material permissible to share with a third-party service. When the information is not allowed, summarise the concept in your own words.
- Be transparent by explaining that you designed the simulator and an AI wrote the code. It models transparent AI use and opens a useful discussion about the boundary between directing a tool and delegating judgement to it.
- Check your own institution's rules on generative AI in teaching, on where teaching material may be hosted, and on what must be disclosed.
- Ask for readable fonts, high contrast and keyboard-navigable controls in Phase 4 and check the result on a small screen. Have an alternative activity ready for any student who cannot use the simulator.

9.2 Common troubleshooting problems

Problem	What to do
The simulator will not run offline.	"Inline every library, font, image and asset so this one HTML file runs with no internet connection." Then test with Wi-Fi off.
The scores do not add up.	Describe the exact wrong behaviour and the expected behaviour, and ask for that fix only. Give a worked example with numbers.
Content is factually wrong.	Point to the specific claim, supply the correct fact, and ask the AI to correct it and check every related value it feeds into.
It is too easy or too hard.	Ask to adjust the starting budget, the frequency of random events or the scoring thresholds. Change one at a time and re-test.
One strategy always wins.	"Adjust the scenarios so each option is the best choice in at least two rounds, and tell me which rounds favour which option."
Students finish at very different speeds.	Add an optional hard mode or a second scenario for fast finishers, and give slower students a partner.
The AI rewrites unrelated parts.	"Change only [X] and keep everything else identical." Revert to your last saved version and make the change again, on its own.
The AI keeps reintroducing a fault you fixed.	The conversation is too long. Start a new chat and hand over the file using the prompt in Section 6.4.
The game breaks on phones.	"Make the layout responsive so every control and all text fit on a 6-inch phone screen in portrait orientation."
Students see a blank page after downloading.	The file was probably saved as .txt. Rename it so it ends in .html.

10. ADAPTING ADEPT

As the ADEPT framework was designed with flexibility and scalability in mind, below are several examples of how the framework can be used for several disciplines:

Discipline	Simulator idea	Student decision
Marketing	Campaign allocation simulator	Distribute budget across channels and segments
Accounting and Finance	Cash-flow survival simulator	Choose which obligations to pay and when
Operations	Production line balancing simulator	Assign staff and machines to workstations
Hospitality and tourism	Front-desk or event-planning scenario	Handle bookings, overbooking and guest complaints
Human resources	Recruitment and retention simulator	Select candidates and set the reward package
Engineering and STEM	Parameter-tuning laboratory	Adjust variables and observe simulated outcomes
Health sciences	Triage or care-planning scenario	Prioritise patients against changing information
Law and policy	Case-strategy or drafting scenario	Choose arguments and concessions in sequence
Humanities and Social science	Choice-based scenario over time	Make decisions whose consequences appear several turns later

APPENDIX A - PHASE PROMPT SHEET

A1. Phase 1 - Architect input

PROMPT TEMPLATE

I want you to play the role of a game designer. I am thinking of making an HTML-based simulator-style game that focuses on **[learning objective]** *for* **[student level]**. *This simulator focusses on* **[scenario]** *where players will* **[brief game mechanic]**. *Suggest to me how this can be designed, the detailed mechanics, core gameplay and scoring details. Use the* **[lesson materials]** *provided as the information to be integrated into the gameplay and details. Do not generate any code yet but give me the detailed mechanics, core gameplay and scoring details for me to review. Ask me if there are revisions to be made and once I give you the approval to generate the actual code, generate it in canvas.*

A2. Phase 2 – Developer build

PROMPT TEMPLATE

The proposed game overview and details are ok. Turn this into a playable HTML offline game using canvas that is suitable to be played on ***[devices]***.

Build it as a single self-contained .html file. Put all the code, styling and content inside that one file and do not link to anything on the internet, so it runs with no connection. Use clear labels and readable font sizes for ***[student level]***. *Show me the working game in canvas when it is ready.*

A3. Phase 3 — Review and testing

PROMPT TEMPLATE

In **[where in the game]**, *the current behaviour is* **[what happens now]**. *It should be* **[what should happen]**.

Please fix only this. Do not change any other mechanic, the scoring weights, the layout or the wording anywhere else in the game. After the fix, tell me in one sentence what you changed.

A4. Phase 4 – Requesting refinements

PROMPT TEMPLATE

How to improve the game mechanics and game logic so that it better matches the learning objective and learning materials? Provide some suggestions on how to improve them for me to review and choose which to implement into the game. I will let you know which suggestion to be integrated into the game.

A5. Phase 4 – Continuing in a fresh conversation

PROMPT TEMPLATE

I have the following HTML game attached which I want you to refine. The complete code is below. Read it and confirm you understand the game's rules and scoring before changing anything.

This game teaches **[learning objective]** *to* **[student level]**. *The scoring works as follows:* **[one-line summary]**.

Do not redesign or rewrite it. I will give you one change at a time, and you will implement only what I ask, keeping everything else identical. First, confirm what you understand about the game.

[Upload the latest revision of your saved .html file here]

APPENDIX B - PEDAGOGICAL ACCURACY CHECKLIST

Pedagogy Accuracy Review Checklist

To systematically evaluate whether the AI-generated simulator aligns with pedagogical intent and expectations.

SUBJECT CODE & NAME	PROGRAMME / LEVEL	SIMULATOR NAME	REVIEW DATE

LEARNING OBJECTIVE — LO1	LEARNING OBJECTIVE — LO2

How to Use — Rate each criterion from ① Not Aligned to ⑤ Fully Aligned.

A	SECTION A · CONTENT ACCURACY						WEIGHT 35%
CRITERIA		1	2	3	4	5	SCORE
Conceptual accuracy aligns with syllabus		①	②	③	④	⑤	
Appropriate depth for student level		①	②	③	④	⑤	
Correct use of terminology		①	②	③	④	⑤	
Game rules reflect real-world principles		①	②	③	④	⑤	
SECTION A AVERAGE SCORE							

B	SECTION B · LEARNING OBJECTIVE ALIGNMENT						WEIGHT 40%
CRITERIA		1	2	3	4	5	SCORE
All LOs are represented in gameplay		①	②	③	④	⑤	
Game mechanics operationalise LOs		①	②	③	④	⑤	
Scoring reflects LO achievement		①	②	③	④	⑤	
Matches intended taxonomy level		①	②	③	④	⑤	
SECTION B AVERAGE SCORE							

C	SECTION C · GAME VARIABLES						WEIGHT 25%
CRITERIA		1	2	3	4	5	SCORE
Realistic business scenario		①	②	③	④	⑤	
Variables appropriate for student level		①	②	③	④	⑤	
Logical and consistent gameplay		①	②	③	④	⑤	
Engagement does not compromise accuracy		①	②	③	④	⑤	
SECTION C AVERAGE SCORE							

FINAL SCORING			
Section	Avg	Weight	Weighted
Section A. Content Accuracy		0.35	
Section B. LO Alignment		0.40	
Section C. Game Variables		0.25	
FINAL SCORE			

SIMULATOR QUALITY GUIDE	
Score	Simulator Quality
> 4.5	● Excellent — Ready for Phase 4
> 4.0	● Good — Minor refinement
> 3.5	● Moderate — Revision needed
< 3.5	● Poor — Redesign required

KEY STRENGTHS	KEY ISSUES IDENTIFIED	RECOMMENDED REVISIONS

APPENDIX C - GLOSSARY

Term	Definition
Artefact / Artifact	The name Claude gives to its live preview panel.
Blueprint	The written design the AI produces in Phase 1, before any code exists.
Canvas	The panel in ChatGPT or Gemini that shows the working game beside the conversation.
Generative AI (GenAI)	A tool that produces text, images or code in response to written instructions.
Hallucination	Content an AI produces that is fluent, confident and wrong. The reason Phase 3 exists.
HTML file	A file ending in .html that a browser can open. In ADEPT it holds the entire simulator.
LMS	Learning management system: Moodle, Canvas, Google Classroom, Blackboard and similar.
Offline / self-contained	The file works with the internet switched off because everything it needs is inside it.
Prompt	The instruction you type to the AI.
Responsive layout	A design that rearranges itself to fit a phone screen as well as a laptop.
TPACK	A framework describing the technological, pedagogical and content knowledge teaching requires (Mishra & Koehler, 2006).
Version	A saved copy of the file at a point in time, for example Logistics_Tycoon_v3.html.

APPENDIX D - FAQ

Do I need a paid AI subscription?

No, but free tiers restrict how many long messages you can send in a day, and a build uses many. Expect a first simulator to be difficult to finish in one sitting on a free plan.

Will the AI keep my simulator or my teaching material?

Policies differ between providers and change over time. Check the terms of the tool you use, prefer material you are entitled to share, and keep student data out of prompts entirely.

How long does the second simulator take?

Roughly half the time of the first. Most of the learning curve is in Phase 3, and once you know what to test for, the review moves quickly.

Can students build their own simulators?

Yes, and it works well as a project. The same six phases apply, and the Pedagogical Accuracy Checklist becomes the marking rubric, with students defending their design decisions rather than their code.

What if my institution restricts AI tools?

Check the policy before starting. Some institutions provide an approved tool; others restrict what may be uploaded. The method works with any tool that can produce code and show a preview.

Do I need to know any code to fix problems?

No. Every fix in this guide is a sentence in plain language describing what happens and what should happen instead. If you find yourself editing code, the fault report was not specific enough.

CITATION AND CONTACT

Rangel, V. A., Shaik Saffarudin, M. R., Tan, M. M. J., & Mohd Khairul Fauzi, M. F. N. (2026). *ADEPT: AI-Developed, Educator-Directed Pedagogical Transformation. Teacher's manual and replication guide* (Version 1.0). Sunway College Johor Bahru.

Prompt templates, the Pedagogical Accuracy Checklist and a sample simulator are available at https://github.com/vincentarsunway/ADEPT-.

Questions and reports of how the framework worked in your own classroom, by contacting the corresponding author: vincentr@sunway.edu.my.

REFERENCES

Kolb, D. A. (1984). *Experiential learning: Experience as the source of learning and development*. Prentice-Hall.

Mishra, P., & Koehler, M. J. (2006). Technological pedagogical content knowledge: A framework for teacher knowledge. *Teachers College Record, 108*(6), 1017–1054.

Mishra, P., Warr, M., & Islam, R. (2023). TPACK in the age of ChatGPT and generative AI. *Journal of Digital Learning in Teacher Education, 39*(4), 235–251.

Prensky, M. (2001). *Digital game-based learning*. McGraw-Hill.